

Day 1 — Foundations: Full Instructor Teaching Material

Total session time: 210 minutes (3.5 hrs). Each section below is sized to its time budget with talking points, examples, board work, and transitions — enough material to actually fill the block, not just bullet prompts.

1. PLC Basics — 15 min

Opening hook (2 min): Start with a question to the room: "How many of you have flipped a light switch and had it do something more complicated than just turning on a light — like a timer, or a motion sensor override?" Use their answers to bridge into: that complexity, at industrial scale, is what a PLC manages.

Definition (3 min): A PLC (Programmable Logic Controller) is an industrial computer built to control machinery — motors, valves, conveyors, actuators — by continuously reading inputs (sensors, buttons, limit switches) and driving outputs (lamps, motors, solenoids, valves) according to a stored program. Write this on the board as a loop diagram: **Inputs → Logic → Outputs → repeat.**

Why it replaced relay panels (4 min): Before PLCs, every piece of industrial logic was hardwired using physical relays — a rack of electromechanical switches bolted together with actual wire. If you wanted to change *any* behavior — say, add a delay before a conveyor starts — someone had to physically rewire the panel. That could take hours, required the machine to be shut down, and any mistake meant re-tracing wires by hand.

A PLC replaces that wiring with software logic. The physical inputs and outputs stay the same — buttons, sensors, motors are still wired to terminals — but the "brain" that decides what happens is now a program you can edit in minutes from a laptop, without touching a single wire.

Analogy (2 min): "Imagine your house wiring, but instead of physically rewiring a switch every time you want the light to behave differently — say, blink twice before turning on — you just change one line of code." Relays = hardware logic. PLC = software logic, same physical I/O underneath.

Real-world use cases (2 min) — list on board, one line each:

- Conveyor belts — start/stop, jam detection
- Packaging lines — fill, seal, count
- Elevators — floor sequencing, door interlocks
- Traffic lights — sequencing (ties directly into Day 2's project — flag this connection explicitly so it lands as forward-referenced, not a random example)

Key terms to define (1.5 min):

- **Input** — a signal coming into the PLC from the real world (a sensor, a button)
- **Output** — a signal the PLC sends out to control something (a motor, a lamp)

- **Program/Logic** — the set of instructions deciding how outputs respond to inputs
- **Scan** — just the word for now; full explanation comes in Block 3. Say explicitly: "Hold that word, we'll unpack it properly in about 40 minutes."

Common misconception to preempt (0.5 min): Students often think a PLC is "just a computer." Correct this directly: it's ruggedized for industrial environments — vibration, heat, electrical noise — and it runs *one* program in a continuous, predictable loop. It is not a general-purpose OS juggling multiple arbitrary programs like a laptop does.

2. PLC Hardware — 30 min

2a. CPU & Power Supply — 8 min

The **CPU** is where the program actually executes. Every scan, it reads inputs, solves the logic, and writes outputs. Draw a simple block diagram: Power Supply → CPU ↔ (Input Module | Output Module).

The **power supply** converts incoming AC (110V or 230V, depending on region) into the DC voltage — typically 24V — that powers the CPU and I/O modules.

Point out explicitly: most industrial PLCs run 24V DC specifically on the I/O side because it's low enough to be safer to touch than mains voltage, and it's a de facto industry standard — meaning sensors, relays, and modules from different manufacturers are generally compatible with each other on 24V.

Ask the class: "Why not just run everything at 230V AC directly?" Let them guess, then confirm: safety and standardization — 24V DC won't seriously shock someone troubleshooting a live panel, and it lets you mix vendors' components freely.

2b. Input Modules — 8 min

Input modules convert real-world signals — a button press, a sensor's state — into a form the CPU can actually read and act on.

Introduce **digital vs. analog** at naming level only — tell the class explicitly: "Analog gets its own full day later this week, so today we're just planting the label."

- **Digital input:** reads a signal as either ON or OFF — no in-between.
- **Analog input:** reads a continuous range of values (e.g., a temperature between 0–100°C). Not covered in depth today.

Explain the electrical convention for digital inputs: a digital input module typically reads 24V DC as logical "1" (true) and 0V as logical "0" (false). Important nuance to state clearly: the module doesn't know your *intent* — whether "1" means "pressed" or "not pressed" depends entirely on how you wired the switch (NO or NC — covered in Block 4). The module just reports voltage; the meaning is assigned by wiring and by the programmer.

2c. Output Modules — 8 min

Three common output types — name all three, since students will encounter each on real jobs even if today's exercises only use one:

Type	Switches	Speed	Notes
Relay	AC or DC	Slower	Mechanical, robust, electrically isolates the PLC from the load
Transistor	DC only	Fast	No mechanical wear — good for high-frequency switching
Triac	AC only	Solid-state	Common for lamps and solenoids

Frame the takeaway plainly: "If someone hands you a spec sheet on the job and it says 'transistor output module,' you now know that module can't drive an AC contactor directly — you'd need a relay in between." This is the kind of fact that saves someone from burning out a module in the field.

2d. Label Exercise — 6 min

Hand out a printed PLC diagram. Have students physically point to and label:

- CPU
- Power supply
- Input terminal block
- Output terminal block

Keep this deliberately hands-on and low-stakes — the goal is building spatial familiarity with real hardware before anyone wires anything for real later in the day. Walk the room while they work; don't lecture during this — let it be quiet, physical, and self-paced.

3. PLC Scan Cycle — 15 min

Frame the stakes up front (1 min): Tell the class directly: "This next concept is the single most important mental model for debugging PLC programs all week. If this doesn't land now, half of Day 3 and Day 4's debugging exercises will feel like magic instead of logic."

The core concept (6 min): A PLC doesn't execute code line-by-line and stay there the way a general-purpose program does. It runs a continuous loop, typically completing in single-digit milliseconds:

1. **Input scan** — read the current state of every physical input and store it in memory (the *input image table*). Critical detail: inputs are **not** re-read mid-logic — they're frozen for the entire cycle.
2. **Logic solve** — execute the entire ladder program top to bottom, using the frozen input values from step 1, writing results to an internal *output image table* — not the physical outputs yet.
3. **Output update** — write the output image table to the actual physical output terminals, all at once.

Then it repeats, indefinitely, for as long as the PLC is running.

Why scan time matters (2 min): A slow scan means slower response to real-world events. If a machine has a fast-moving sensor event and the scan cycle is too slow, the PLC can literally miss it — the input state can change and change back between two scans and the PLC never sees it happened.

Why this matters for debugging — say this explicitly (3 min): Every weird bug where "the output didn't update when I expected" traces back to not understanding *when* inputs are actually read versus *when* logic uses them. If a button changes state mid-scan, the PLC won't see that change until the *next* scan cycle — not instantly. This single fact resolves a huge fraction of "why isn't my program working" confusion later in the week.

Whiteboard walkthrough (3 min): Draw one simple rung — a single NO contact driving a coil. Trace it through all three phases as a class:

- Phase 1: contact state read as "0" or "1", frozen
- Phase 2: logic solved using that frozen value, coil result computed
- Phase 3: coil result written to the physical output terminal

Physically point at each phase on the board as you narrate it, so the "freezing" behavior is visually anchored, not just described in words.

4. Inputs/Outputs + NO/NC Contacts — 40 min

4a. Wiring a Push-Button Input — 10 min

Physically wire a push button into an input terminal in front of the class (or have students do it in pairs at their stations).

Cover:

- Terminal numbering convention on the module (walk the row, point at labels)
- Common wire vs. signal wire — clarify which terminal is the shared reference and which carries the actual signal
- **Verification step, stated explicitly:** before assuming the wiring is correct, check the PLC software's I/O status view to confirm the input actually toggles when the button is pressed. Frame this as the professional habit to build now: never assume a wire is good — verify it in software.

4b. Wiring a Lamp Output — 10 min

Same process, applied to an output: wire a lamp or LED indicator to an output terminal.

Confirm the module type matches the load — reference back to the output types table from Block 2c. Ask the class: "If this is a transistor output module, could I wire an AC lamp to it?" (No — DC only.) This reinforces the earlier hardware content instead of treating it as a one-off fact.

4c. Live Status Monitoring — 10 min

Show the online status view in the PLC software — the live green/red or highlighted indication of which inputs and outputs are currently active.

State plainly: "This is the single most useful debugging tool you'll use all week." Make sure every student navigates to it themselves at least once, unprompted, before moving on — this isn't a watch-me-do-it moment, it's a everyone-does-it-once moment.

4d. NO vs. NC in Real Circuits — 10 min

Define both clearly, with a simple diagram for each:

- **NO (Normally Open):** contact is open (no connection) at rest, closes when actuated. Logic reads "0" until pressed.
- **NC (Normally Closed):** contact is closed at rest, opens when actuated. Logic reads "1" until pressed.

Why E-stops are always NC — the important safety concept to land clearly (state this plainly, don't rush it):

If an E-stop is wired NC and someone cuts the wire, unplugs the connector, or the switch itself fails, the circuit reads as "pressed" (open) — and the machine stops. A wiring fault **fails safe**.

If it were wired NO instead, a cut wire would look *electrically identical* to "not pressed" — meaning a real safety fault could go completely undetected while the machine keeps running.

Emphasize: this is a genuine industrial safety standard, not a style preference or convention someone made up. Consider asking the class to explain it back in their own words to confirm it landed before moving on.

5. Ladder Logic Basics + Tag Naming — 60 min

5a. Rungs & Basic Instructions — 15 min

Define a **rung**: one horizontal line of logic in a ladder diagram — conceptually similar to one line of code, but drawn as a circuit. Contacts (conditions) sit on the left, a coil (output) sits on the right. Power "flows" left to right if the conditions are true — draw this left-to-right flow explicitly with an arrow on the board.

Cover the three basic logical operations with a rung diagram for each:

- **AND** — two NO contacts in series. Both must be true for the coil to energize.
- **OR** — two NO contacts in parallel (branches). Either being true energizes the coil.
- **NOT** — not a separate symbol; represented by using an NC contact instead of NO for that condition.

Draw all three on the board side by side so students can visually compare series vs. parallel vs. NC-substitution in one glance.

5b. Guided Practice — 15 min

Build 2–3 simple rungs together as a class *before* releasing students to work independently. State the reasoning behind this out loud: this is deliberately extra time compared to

jumping straight to solo work, because ladder logic syntax is genuinely unfamiliar to first-timers, and guided repetitions here prevent a lot of frustrated, directionless debugging later.

Suggested progression:

1. Single NO contact → coil (simplest possible rung)
2. Two NO contacts in series (AND)
3. Two NO contacts in parallel (OR)

Narrate your own thought process out loud while building each one — don't just produce the finished rung silently.

5c. Symbolic Addressing & Tag Naming — 15 min

Explain the difference between:

- **Raw addressing** (e.g., `I0.0`, `Q0.1`) — the literal physical terminal address
- **Symbolic tags** (e.g., `Start_PB`, `Motor_Run`) — a human-readable name mapped to that address

State the professional norm directly: production programs are almost never written with raw addresses visible in the logic — they use tags, because six months later nobody remembers what `I0.3` was physically wired to.

Establish a naming convention now that will be used consistently for the rest of the week — pick one and commit to it as a class, e.g.:

- PascalCase (`MotorRun`) or snake_case (`motor_run`)
- Verb/noun ordering convention (e.g., `Start_PB` for a start pushbutton, `Motor_Run` for a run output)

Write the chosen convention on the board and treat it as a standing rule from this point forward.

5d. Commenting Rungs — 5 min

Show how to add a rung comment/description in the software.

Frame the distinction for students to remember: "The rung shows you *what* the logic does; the comment tells you *why*." A one-line comment explaining intent saves enormous debugging time for someone else — or future-you — reading the program weeks later.

5e. Project: Two-Button AND/OR Logic — 10 min

Students build:

1. A two-button **AND** rung — lamp on only if both buttons are pressed
2. Convert it to an **OR** rung — lamp on if either button is pressed

Requirement to enforce while circulating: tags must be used throughout, not raw addresses — this is the first graded application of the naming convention established in 5c, so hold the

line on it even under time pressure.

6. Day Project — Motor Start/Stop with Seal-In Circuit — 50 min

Core concept to land before touching software (state this explicitly at the start of the block): A momentary push button (Start) needs to turn on a motor that *stays* on after the button is released. This requires the output to "seal itself in" using its own contact as a self-holding condition, combined with a Stop button (NC, wired so pressing it breaks the circuit) that overrides the seal.

The logic pattern — write this on the board and keep it visible the whole block:

```
(Start OR Motor_Run) AND NOT Stop → Motor_Run
```

Explain the mechanism explicitly: the `Motor_Run` contact placed in parallel with `Start` is what creates the seal-in — once energized, the coil's own contact keeps the rung true even after the Start button is released. This is the first "real" industrial logic pattern of the week — treat it with weight; don't rush past it.

Walkthrough structure:

1. Plan — 5 min Sketch the seal-in logic on paper before touching any software. This forces students to reason through the pattern rather than trial-and-error clicking in the PLC software. Circulate and check sketches before letting anyone move to step 2.

2. Wire I/O — 10 min

- Start button — wired NO
- Stop button — wired NC
- Motor output — confirm correct output module type per Block 2c

3. Write the rung — 15 min Implement the seal-in pattern exactly as sketched in step 1. Encourage students to reference their own paper sketch rather than someone else's screen.

4. Test & debug — 15 min Flag the most common failure mode explicitly, before students even hit it: forgetting the seal-in contact entirely, which results in the motor only running while Start is physically held down. When circulating, if you see this bug, don't just fix it — ask "what's missing from the rung?" and let them trace it back to the missing parallel contact themselves.

5. Class review — 5 min Surface the 2-3 most common mistakes actually observed while circulating, and address them together as a class. This closes the loop on the day — it should feel like a debrief, not a lecture.

If finished early: Add forward/reverse control with interlocking — two motor directions that can never both be active at once. Introduce this as **mutual exclusion logic**: each direction's rung includes a NOT of the other direction's output. Flag explicitly that this same interlocking concept gets reused for safety circuits on Day 5 — this is deliberate forward-seeding, not a throwaway bonus exercise.

